

EAGLE Document Schema

Four schema layers for each doc type.

PRESET

DOC TYPE

CORE (BACKEND-GENERATABLE)

■ SOW ●

■ pws

■ igce ●

■ market_research ●

■ acquisition_plan ●

■ justification ●

eval_criteria

security_checklist

section_508

■ cor_certification

contract_type_justification

■ son_products

■ son_services

purchase_request

MICRO-PURCHASE

price_reasonableness

required_sources

TEMPLATE / FORM

■ subk_plan

subk_review

■ buy_american

■ conference_request

conference_waiver

bpa_call_order

gfp_form

srb_request

quotation_abstract

receiving_report

technical_questionnaire

promotional_item

exemption_determination

mandatory_use_waiver

reference_guide

FRONTEND-ONLY (NO BACKEND SUPPORT)

funding_doc

d_f

qasp

source_selection_plan

sb_review

human_subjects

DISPLAY

■ Has docx/xlsx template

● Has typed Pydantic model

Others fall back to BaseDocumentData with extra="allow"

SOW — Statement of Work

TYPED MODEL

TEMPLATE (DOCX)

CORE

1. Pydantic model

4 EXTRA FIELDS

FIELD	TYPE	NOTES
title	Optional[str]	shared — BaseDocumentData
description	Optional[str]	shared — BaseDocumentData
estimated_value	Optional[float]	shared — BaseDocumentData
period_of_performance	Optional[str]	shared — BaseDocumentData
contract_type	Optional[enum]	shared — BaseDocumentData
acquisition_method	Optional[enum]	shared — BaseDocumentData
competition	Optional[str]	shared — BaseDocumentData
deliverables	Optional[list[str dict]]	specific to sow
tasks	Optional[list[str dict]]	specific to sow
place_of_performance	Optional[str]	specific to sow
security_requirements	Optional[str]	specific to sow

All fields are Optional. "Required" is enforced at the template and UI layer, not by Pydantic.

2. Template placeholder map

7 PLACEHOLDERS

statement-of-work-template-eagle-v2.docx

DATA FIELD	PLACEHOLDER
title	→ {{PROJECT_TITLE}}
description	→ {{DESCRIPTION}}
period_of_performance	→ {{PERIOD_OF_PERFORMANCE}}
deliverables	→ {{DELIVERABLES}}
tasks	→ {{TASKS}}
place_of_performance	→ {{PLACE_OF_PERFORMANCE}}
security_requirements	→ {{SECURITY_REQUIREMENTS}}

3. Field name aliases

AI/USER INPUT → CANONICAL

AI / user input is normalized via `normalize_and_validate_document_payload()` before it reaches the model.

ALIAS	CANONICAL
<code>competition_type, competition_strategy, full_open</code>	→ <code>competition</code>
<code>contract_period, duration, pop, performance_period</code>	→ <code>period_of_performance</code>
<code>estimated_cost, budget, total_value</code>	→ <code>estimated_value</code>
<code>total_cost</code>	→ <code>total_estimate</code>
<code>requirement, requirement_description, objective, requirement_summary, scope</code>	→ <code>description</code>

Show me how to build a valid `create_document` payload for an EAGLE "sow" (Statement of Work).

Key facts: the typed sow model extends `BaseDocumentData` with: `deliverables`, `tasks`, `place_of_performance`, `security_requirements`; the docx template "statement-of-work-template-eagle-v2.docx" needs these fields populated: `title`, `description`, `period_of_performance`, `deliverables`, `tasks`, `place_of_performance`, `security_requirements`; the shared base fields are `title`, `description`, `estimated_value`, `period_of_performance`, `contract_type`, `acquisition_method`, `competition`; remember that AI input goes through `normalize_and_validate_document_payload`, so aliases like "requirement" → "description" and "pop" → "period_of_performance" are resolved automatically.

Reference files: `server/app/ai_document_schema.py` (canonical enums + Pydantic models), `server/app/template_registry.py` (placeholder_map), `server/app/template_schema.py` (section-level schema).

Give me a minimal and a fully-populated example payload, and explain which fields are effectively required (i.e., needed by the template or the frontend) vs. truly optional.